

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models
GitHub Repository:	https://github.com/sangee-3010/Machine_Learning
Student Name	Sangeetha S K
Register Number	3122247001054
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	23/08/2026

1 Objective

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging and Boosting classifiers.
- To build a Stacked Ensemble model using multiple base learners.
- To compare ensemble models in terms of accuracy, stability, and generalization.
- To analyze the effect of ensemble methods on bias and variance.

2 Problem Statement

Breast cancer diagnosis from digitized fine-needle-aspirate cell measurements can be framed as a binary classification problem. **Input:** 30 numerical features describing the mean, standard error, and worst-case values of cell-nucleus measurements (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension). **Output:** a binary diagnosis label – Benign (0) or Malignant (1). **Objective:** implement and compare three ensemble strategies (Bagging, Boosting, Stacking) on this task, tune each with cross-validated hyperparameter search, and determine which ensembling strategy generalizes best to unseen test data.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC) Dataset
Dataset Source	UCI Machine Learning Repository – https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
Number of Samples	569
Number of Features	30 numerical attributes (id and diagnosis dropped from the feature matrix X)
Number of Classes	2 – Benign (357 samples) and Malignant (212 samples), a moderate 63:37 imbalance
Missing Values	0
Train-Test Split	80% / 20% (455 train / 114 test), stratified on the target label

4 Exploratory Data Analysis

- Dataset overview: `df.shape`, `df.head()`, feature/target separation
- Statistical summary and missing-value check (`X.isnull().sum()`)
- Class distribution of Benign vs. Malignant
- Correlation matrix across all 30 features
- Feature distribution examined via the correlation structure (see below)

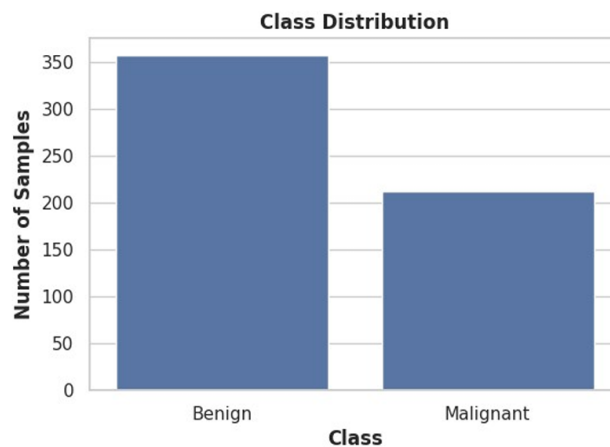


Figure 1: Class Distribution – Benign vs. Malignant

Inference: Of the 569 samples, 357 (62.7%) are Benign and 212 (37.3%) are Malignant. The imbalance is moderate rather than severe, but is still large enough that accuracy alone would understate how well a model detects the minority (Malignant) class. This is why Precision, Recall, and F1-score on the Malignant class are reported alongside accuracy in Section 10. The split is also why the train/test partition was stratified, so both sets preserve this 63:37 ratio.

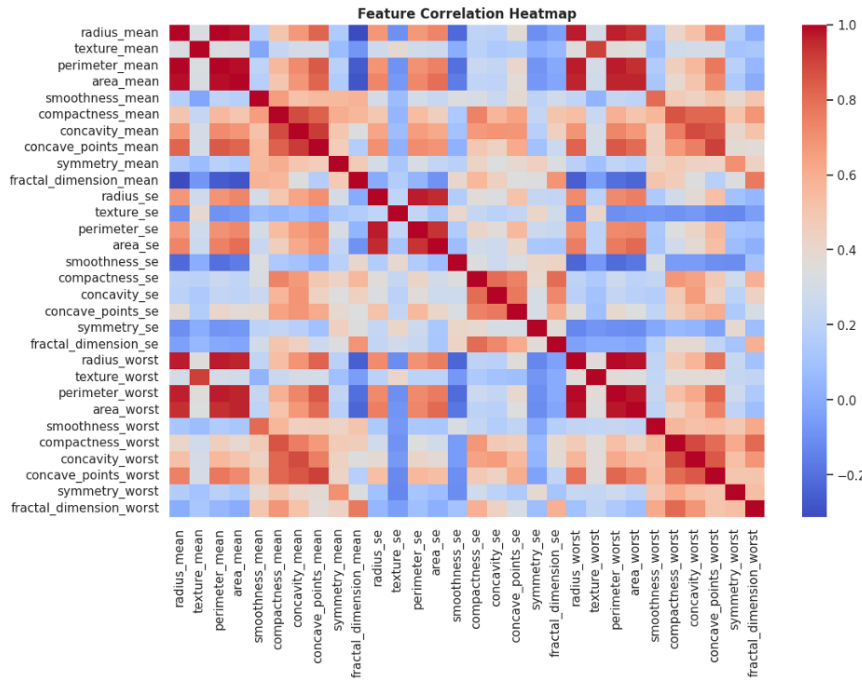


Figure 2: Feature Correlation Heatmap

Inference: The heatmap shows very strong positive correlation among the size-related features (radius, perimeter, area, across the `_mean`, `_se`, and `_worst` groups), which is expected since perimeter and area are geometrically derived from radius. `concavity` and `concave_points` are also strongly correlated with each other and with size features, while `smoothness` and `fractal_dimension` are comparatively independent of the rest. This redundancy suggests tree-based ensembles (robust to correlated features) are a good fit, and that a linear model would benefit more from dimensionality reduction than the ensembles used here.

5 Data Preprocessing

- **Missing value handling:** Not required – `X.isnull().sum().sum()` confirmed 0 missing values across all 30 features.
- **Encoding:** Only the target required encoding; `diagnosis` was mapped from `{'B', 'M'}` to `{0, 1}` with `y = df["diagnosis"].map({"B": 0, "M": 1})`. All 30 input features were already numeric, so no categorical feature encoding was needed.
- **Feature scaling:** **Not applied** in this experiment – `StandardScaler/MinMaxScaler` were not used before fitting the models. This is a reasonable simplification for the Decision-Tree, Bagging, and AdaBoost estimators (which are scale-invariant), but it is a genuine limitation for the Stacked Ensemble, since its `SVC` base learner is scale-sensitive; scaling the features would likely improve the SVM component's contribution to the stack (see Section 12).
- **Train-test split:** 80/20 split via `train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)`, giving 455 training and 114 testing samples with class proportions preserved.

- **Feature engineering:** Not performed; the 30 provided measurements were used directly as the feature set.

6 Machine Learning Algorithm

Bagging (Bootstrap Aggregation)

Working principle: Trains multiple Decision Trees independently, each on a different bootstrap resample of the training data, and aggregates their predictions by majority vote.

Mathematical formulation: for B bootstrap samples and base learners $h_1, \dots, h_B$, the ensemble prediction is

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_B(x)\}.$$

Advantages: reduces variance; suitable for high-variance base learners like unpruned Decision Trees; trees train independently and can be parallelized. **Limitations:** does not reduce bias – if every bootstrap tree shares the same systematic error, averaging does not remove it; less effective when base learners are highly correlated.

Boosting (AdaBoost)

Working principle: Trains estimators sequentially; after each round, misclassified training samples receive higher weight so the next estimator focuses on the harder cases.

Mathematical formulation: the final prediction is a weighted vote of T weak learners,

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right), \quad \alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right),$$

where ϵ_t is the weighted error rate of learner h_t . **Advantages:** reduces bias; often achieves higher accuracy than Bagging on well-separated data; naturally emphasizes hard-to-classify samples. **Limitations:** sequential training cannot be parallelized; sensitive to noisy data and outliers, since misclassified noisy points keep gaining weight.

Stacked Ensemble

Working principle: Combines heterogeneous base models (SVM, Naive Bayes, Decision Tree) whose out-of-fold predictions become the input features for a meta-learner (Logistic Regression), which learns how to optimally weight each base model. **Mathematical formulation:** for base learners $h_1, \dots, h_k$ and meta-learner g ,

$$\hat{y} = g(h_1(x), h_2(x), \dots, h_k(x)).$$

Advantages: can outperform any single base model if the base learners make complementary errors; flexible – any classifier can be a base learner or meta-learner. **Limitations:** more complex and slower to train/tune than Bagging or Boosting; benefits are not guaranteed – as this experiment shows, poorly scaled or non-complementary base learners can make the stack underperform simpler ensembles (Section 12).

Configuration used in this experiment:

- **Bagging:** Base estimator Decision Tree; best configuration 100 estimators, `max_samples=1.0`, `max_features=0.7`.
- **Boosting:** AdaBoost; best configuration 50 estimators, learning rate 1.0.
- **Stacking:** Base models SVM (`probability=True`), Gaussian Naive Bayes, Decision Tree; meta-learner Logistic Regression (`max_iter=1000`); 5-fold internal cross-validation.

7 Hyperparameter Selection

Hyperparameters were selected via `GridSearchCV` with 5-fold cross-validation, optimizing accuracy, over the search spaces below.

Model	Parameter	Search Space (Selected Value)
Bagging	<code>n_estimators</code>	{10, 50, 100} (100)
Bagging	<code>max_samples</code>	{0.7, 1.0} (1.0)
Bagging	<code>max_features</code>	{0.7, 1.0} (0.7)
Boosting	<code>n_estimators</code>	{50, 100, 150} (50)
Boosting	<code>learning_rate</code>	{0.5, 1.0} (1.0)
Stacking	<code>base_models</code>	SVM, Naive Bayes, Decision Tree (fixed)
Stacking	<code>final_estimator</code>	Logistic Regression (fixed)

Best cross-validation accuracy: Bagging 96.92%, Boosting 96.26% (Stacking was not subjected to grid search; its 5-fold CV accuracy with the fixed configuration above was 94.29%, see Section 9).

8 Hyperparameter Evaluation Results

Bagging Results

`GridSearchCV` best parameters: {'`max_features`': 0.7, '`max_samples`': 1.0, '`n_estimators`': 100}, with best CV accuracy **96.92%**.

Table 2: Bagging Hyperparameter Evaluation (`max_features=1.0` across all rows shown)

<code>n_estimators</code>	<code>max_samples</code>	Avg CV Accuracy (%)	Avg CV F1 Score
10	0.7	94.95	0.9330
10	1.0	95.16	0.9362
50	0.7	94.73	0.9300
50	1.0	95.82	0.9456
100	0.7	95.16	0.9365
100	1.0	95.82	0.9457

Boosting Results

`GridSearchCV` best parameters: {'`learning_rate`': 1.0, '`n_estimators`': 50}, with best CV accuracy **96.26%**.

Table 3: Boosting Hyperparameter Evaluation

n_estimators	learning_rate	Avg CV Accuracy (%)	Avg CV F1 Score
50	0.5	95.82	0.9441
50	1.0	96.26	0.9489
100	0.5	95.60	0.9406
100	1.0	96.04	0.9463
150	0.5	96.04	0.9464
150	1.0	96.04	0.9463

Stacked Ensemble Results

Table 4: Stacked Ensemble Evaluation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
SVM, Naive Bayes, Decision Tree	Logistic Regression	94.29	0.9213

9 Performance Comparison

Table 5: Performance Comparison of Ensemble Models (on held-out test set, $n = 114$)

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging	96.49	1.0000	0.9048	0.9500
Boosting	98.25	1.0000	0.9524	0.9756
Stacked Ensemble	92.98	0.9722	0.8333	0.8974

Best Performing Model: Boosting (AdaBoost), Accuracy = 98.25%.

10 Result Visualization

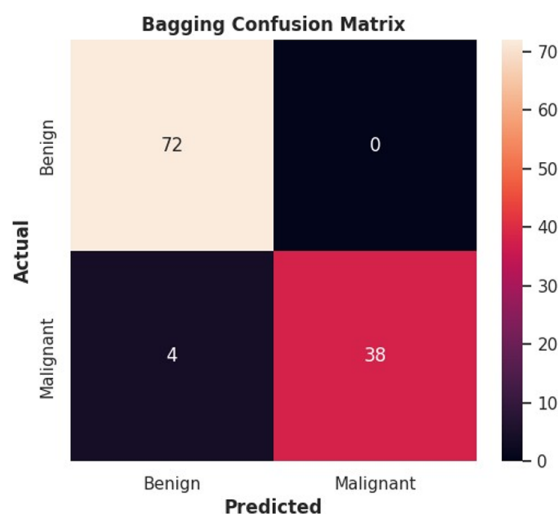


Figure 3: Bagging Confusion Matrix

Inference: The Bagging model correctly classifies all 72 Benign cases but misses 4 of 42 Malignant cases ($\text{Recall} = 0.9048$), reflecting perfect precision but slightly reduced sensitivity toward the minority class. No false positives occur, so every Bagging Malignant prediction was trustworthy, but a handful of true Malignant cases were missed.

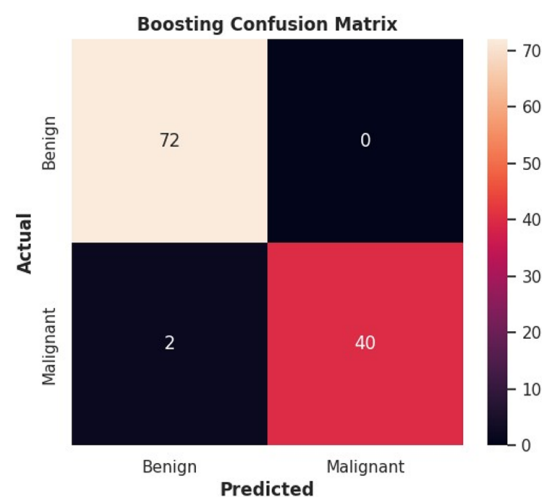


Figure 4: Boosting Confusion Matrix

Inference: The Boosting model correctly classifies all 72 Benign cases and misses only 2 of 42 Malignant cases ($\text{Recall} = 0.9524$), the best balance of precision and recall among the three ensembles. This directly explains its top F1-score (0.9756) and confirms AdaBoost's sequential error-correction paid off most on the hardest borderline cases.

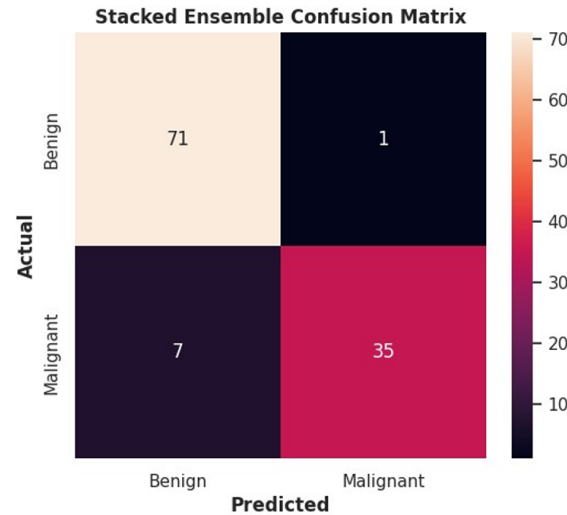


Figure 5: Stacked Ensemble Confusion Matrix

Inference: The Stacked Ensemble shows the weakest Malignant-class recall (0.8333, i.e. 7 of 42 malignant cases missed), suggesting that on this train/test split the unscaled SVM + Naive Bayes + Decision Tree combination, blended via Logistic Regression, was less effective than either single-family ensemble at its default decision threshold.

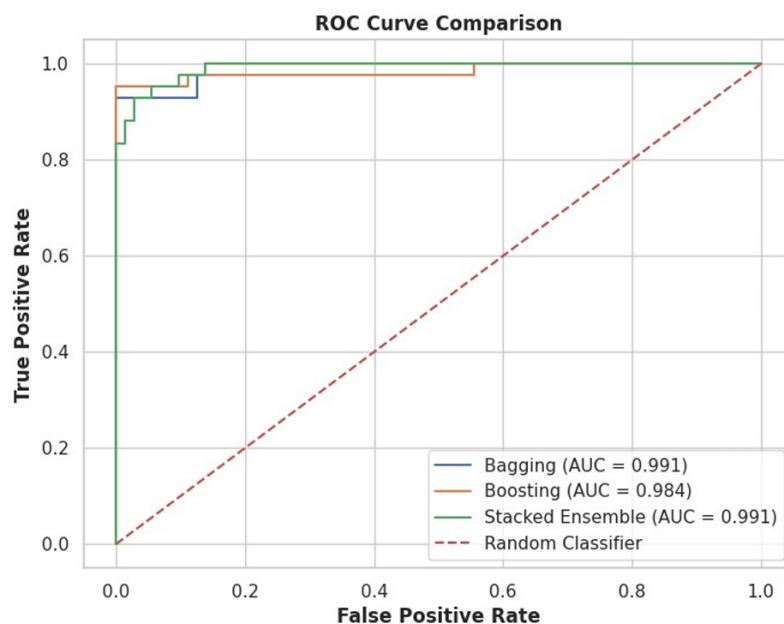


Figure 6: ROC Curve Comparison

Inference: All three models achieve high AUC (Bagging 0.9907, Boosting 0.9841, Stacked Ensemble 0.9911), meaning each ranks Malignant cases above Benign cases correctly in the overwhelming majority of pairwise comparisons. Notably, the Stacked Ensemble has the *highest* AUC despite the *lowest* accuracy/F1 at the default 0.5 threshold – its probability estimates are well-ranked, but the default threshold is not well-calibrated for this data. Threshold tuning would likely close much of the gap with Boosting.

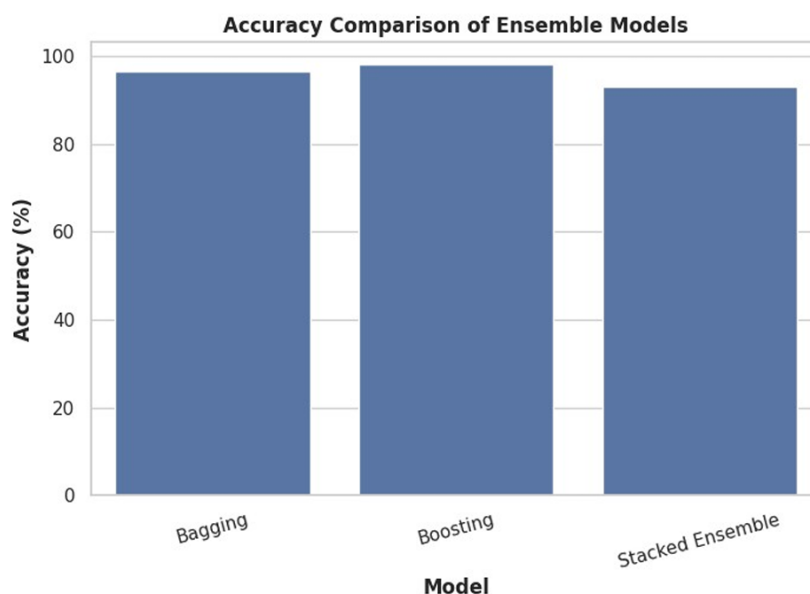


Figure 7: Accuracy Comparison of Ensemble Models

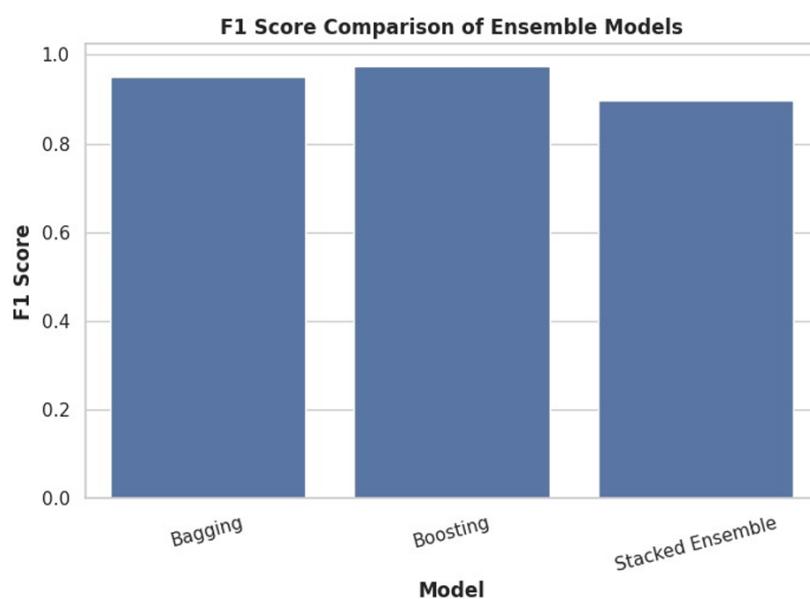


Figure 8: F1 Score Comparison of Ensemble Models

Inference: Both charts rank the three models identically – Boosting > Bagging > Stacked Ensemble – on this test split, reinforcing that sequential error-correction generalized best here, bagging’s variance reduction was a close second, and the stacked combination underperformed relative to its individual base learners’ potential.

11 Discussion

Observations

- Performance was strong across all three ensembling strategies (92.98–98.25% accuracy, $AUC > 0.98$), confirming that the WDBC features are highly informative for this diagnosis task.
- Boosting achieved the best precision/recall balance and the highest F1-score (0.9756), consistent with its sequential, error-focused training.
- Bagging came a close second, with perfect precision but slightly lower recall than Boosting.
- The Stacked Ensemble had the highest ROC-AUC (0.9911) of the three models, yet the lowest accuracy and F1-score at the default 0.5 decision threshold – its probability estimates rank cases well, but the default cutoff is not well-calibrated for this data.

Strengths

- Boosting’s sequential re-weighting directly targeted the harder, borderline cases, giving it the best overall balance of precision and recall.
- Bagging’s independent bootstrap averaging gave perfect precision (zero false-positive Malignant predictions) on the test set.
- All three models generalized well from only 455 training samples, reflecting how strongly correlated the WDBC features are with diagnosis.

Limitations

- Feature scaling was not applied before training. This likely held back the SVM component of the Stacked Ensemble specifically, since **SVC** is scale-sensitive and was trained on raw, unscaled features alongside two scale-invariant learners (Naive Bayes, Decision Tree).
- The Stacked Ensemble’s low accuracy despite its highest AUC points to a probability-calibration issue at the default threshold, not a fundamentally worse model.
- Only a single train/test split (not repeated cross-validation) was used for the final comparison in Section 10, so the reported test-set numbers carry some split-specific variance.

Improvements

- Apply **StandardScaler** to the features before fitting the SVM and Logistic Regression components of the stack.
- Tune the Stacked Ensemble’s decision threshold instead of using the default 0.5 cutoff.
- Extend the Boosting grid search to include **max_depth** of the base Decision Tree stump, since deeper base learners can change AdaBoost’s bias-variance trade-off.
- Report Precision-Recall curves alongside ROC curves, since Recall on the Malignant class is the more clinically relevant metric under class imbalance.

12 Conclusion

Bagging, Boosting, and Stacked Ensemble models were implemented and evaluated on the Wisconsin Diagnostic Breast Cancer dataset. All three ensembles achieved strong performance (92.98–98.25% test accuracy, $AUC > 0.98$), confirming that ensemble methods substantially outperform what a single weak learner would typically achieve on this dataset. Boosting (AdaBoost, `n_estimators=50`, `learning_rate=1.0`) was the best-performing model on this test split, with Bagging close behind and the Stacked Ensemble trailing on default-threshold metrics despite the highest AUC. The experiment demonstrates concretely how Bagging reduces variance through independent averaging, how Boosting reduces bias through sequential error-correction, and how Stacking’s benefit from heterogeneous base learners is real but conditional on the base models being appropriately preprocessed and complementary.

13 References

- [1] Scikit-learn Developers, “Ensemble Methods,” Scikit-learn Documentation. Available: <https://scikit-learn.org/stable/modules/ensemble.html>
- [2] Scikit-learn Developers, “`sklearn.ensemble.BaggingClassifier`.” Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>
- [3] UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Dataset.” Available: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>